

FlowHub

Ein KI-gestützter persönlicher Eingangskorb

Andreas Imboden · CAS AI-Assisted Software Engineering · FFHS FS26

Das Projekt — und die Erfahrung, es mit KI zu bauen



Das Problem — und die Idee

Der digitale Alltag produziert ständig Schnipsel: ein Film, ein Artikel, das Foto einer Quittung.

Heute

Idee → welche App? → öffnen →
kategorisieren → ablegen

5+ Schritte — oft vergessen

Mit FlowHub

Idee → **Telegram** → fertig

1 Schritt — die KI übernimmt

Ein **Telegram-Bot** als einziger Eingang: FlowHub **erkennt** den Input, **kategorisiert** ihn und **routet** ihn automatisch an den richtigen Self-Hosted-Service.

Konkret: das Skill-System

Jeder Input wird einem **Skill** zugewiesen — Erkennung über Keywords, URL-Muster und, wenn nötig, ein **lokales LLM**.

Input	Skill	landet in
heise.de-Artikel	ArticleSkill	Wallabag (read-later) ✓
„The Imitation Game“, share.google/...	MovieSkill	Vikunja (Watchlist) ✓
nichts passt	GenericSkill	Inbox (PostgreSQL) ✓
jellyfin.org „ausprobieren“	HomelabSkill	Wekan (Kanban) · geplant
Foto einer Quittung	DocumentSkill	paperless-ngx (DMS) · geplant



heute live · geplant = Roadmap (gleiche `ISkillIntegration`-Schnittstelle).

Unklar? → Der Bot **fragt mit 2-3 Optionen zurück** (Confidence-Score).

Wiederfinden: semantische Suche

Nicht nur reinwerfen — auch **per Bedeutung wiederfinden**, nicht per Stichwort.

Schritt	Was passiert
Query „alles zu Docker“	→ Embedding (Mistral <code>mistral-embed</code> , 1024 Dim.)
pgvector-Suche	HNSW · Cosine-Distanz · Sub-ms bei < 1 Mio. Zeilen
Treffer	inhaltlich ähnliche Captures — auch ohne exaktes Wort

`GET /api/v1/captures/search?q=...` · Provider per Config tauschbar (OpenAI-kompatibel) ·
ohne Key → sauberes `503`

Tech-Stack

Schicht	Technologie
Backend	.NET 10 / C# / ASP.NET Core (LTS)
Web-UI	Blazor SSR — .NET-native, kein JS-Framework
KI-Integration	Microsoft.Extensions.AI + Ollama (lokal) / Anthropic (Fallback)
Pipeline	MassTransit — In-Memory (dev) / RabbitMQ (prod)
Persistenz	PostgreSQL 17 + pgvector · EF Core 10
Deployment	Docker Compose — 6 Services, Migrations als Init-Container

Inkrementell über 5 Blöcke gebaut: Konzept → UI → Services/KI → Persistenz → Deployment.

Teil 2: Bauen mit KI

Erfahrung · Harness · Learnings

Der Harness – Überblick

Die KI wurde nicht ad-hoc geprompted, sondern über eine **Werkzeugkette** gesteuert:

Ebene	Werkzeug
Agent	Claude Code (interaktiv) · Codex / Copilot ergänzend
Konventionen	<code>ai-instructions</code> (base + <code>dotnet-blazor</code>) → <code>CLAUDE.md</code>
Workflows	eigene Skills: <code>/ui-*</code> , <code>/flowhub-*</code> , <code>/commit</code> , <code>/push</code>
Methode	Brainstorm → Spec → Plan → Subagent → Review
Automatisierung	<code>agent-pipeline</code> (Issue→PR) · <code>examiner-sim</code> (Grading)
Disziplin	Context-Hygiene: Logs-via-File · <code>/clear</code> -Schnitte

Nicht „Prompt rein, Code raus“ — eine Pipeline

Jeder grössere Baustein lief durch denselben strukturierten Ablauf:

Brainstorm → Spec → Plan → Subagent-Implementierung → 2-stufiges Review

1. **Brainstorming** — Design als **A/B/C-Entscheidungen** (z. B. 13 Entscheide für die Async-Pipeline), jede mit Begründung
2. **Spec + Plan** — schriftliches Design, dann **TDD-geordneter** Aufgabenplan
3. **Subagenten** — pro Task ein **frischer Kontext** (Test-First)
4. **Review ×2** — Spec-Konformität, dann Code-Qualität — bevor etwas in `main` geht

Werkzeuge: **ai-instructions** + eigene **Skills**

ai-instructions (eigenes Repo) — Konventionen als **feste Regeln**, nicht als Prompt:

- **base** (stack-agnostisch) + **Stack-Overlay** **dotnet-blazor** → daraus leitet sich **CLAUDE.md** ab
- z. B. **Coding Guidelines** (Clean Code) · **SemVer** · **Conventional Commits** · **12-Factor** · **TDD** — Tests werden nie nachträglich angepasst, nur damit Code grün wird

Eigene CAS-AISE-Skills (Claude-Code-Slash-Commands):

- **/ui-brainstorm** · **/ui-flow** · **/ui-build** · **/ui-review** — der **4-Phasen-UI-Workflow**
- **/flowhub-capture** · **-trriage** · **-issue** — das Produkt selbst bedienen
- **examiner-sim** · **cas-aise-grade-self-check** — **Selbstbewertung** gegen die Moodle-Rubrik

Beispiel: der UI-Workflow

`/ui-brainstorm` → **ASCII-Wireframe** → `/ui-flow`
→ **Mermaid-Flow** → `/ui-build` → `/ui-review`

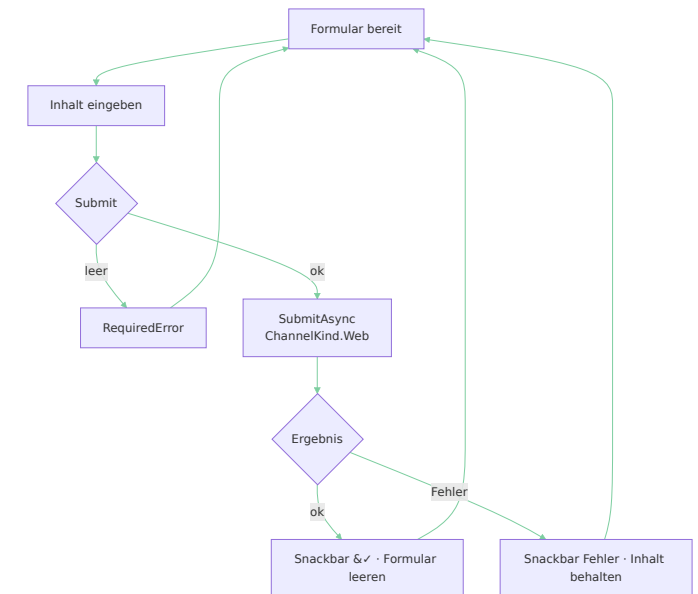
Gate pro Phase — nichts wird gebaut, bevor Wireframe **und** Flow freigegeben sind.

Phase 1 — Wireframe (`New Capture`):

```
FlowHub —
New Capture
  Content * —
    URL / Zitat / Text...
  Skill: [ — KI entscheidet ▼ ]
         [Abbrechen] [Speichern] |
```

Phase 2 — Mermaid-Flow → echtes

`docs/design/new-capture/flow.md`



Context-Hygiene — das unterschätzte Thema

Der Kontext ist das **knappste Gut**. Zwei Disziplinen brachten am meisten:

1 · Logs via File — grösster Token-Fresser waren Console-, Test- und Build-Streams. Statt alles in den Chat: in eine **Datei** schreiben, gezielt mit `Read offset/limit` oder `grep` holen. → **~5-10× weniger Tokens** pro Debug-Session.

2 · /clear-Schnitte — Spec → `/clear` → Plan → `/clear` → Implement.
Jede Phase hinterlässt ein **Artefakt auf Disk**; der Dialog-Ballast wird verworfen.

| *Was zwischen Phasen weiterleben muss, gehört in eine **Datei** — nicht in den Chat.*

Automatisierung — KI prüft KI

agent-pipeline (GitHub Actions) — **autonome Issue-Implementierung**:
Issue mit **ai-implement** labeln → **Branch + Draft-PR**, mit Retry-Policy.

examiner-sim (Multi-Agent-Workflow) — baut die Abgabe-PDFs, **benotet** sie
gegen die **Moodle-Rubrik** mit einem Agenten-Panel und übt die Live-Demo.

*Der Mensch schreibt nicht mehr jede Zeile — er definiert die **Leitplanken**
und lässt **KI-gestützte Prüfungen** finden, was die KI übersieht.*

Wo die KI glänzt — und wo nicht

Über alle Blöcke: **~85-95 % des Codes KI-generiert** — der Mensch-Anteil klein, aber **hochwertig**.

Glänzt — repetitiv & gut spezifiziert

7 `IEntityTypeConfiguration<T>`, EF-Migrations, Refit-Interfaces, CI-YAML;

16 Integrationstests gegen echtes PostgreSQL — **alle grün beim ersten Lauf**.

Scheitert — wo Domäne & Performance zählen

- **N+1-Blindheit** — `ListAsync` ohne `.Include`
- **CASCADE überall** — owned vs. referenced ist eine **Domänen-Entscheidung**
- **Veraltete Versionen** — Trainingsdaten hinken neuen Releases hinterher
- **Feature-Drift** — Scope-Disziplin muss vom **Menschen** kommen

Der Smoke-Test-Moment

Die KI schrieb den ganzen Deployment-Stack. Dann lief **ein** Befehl:

`make smoke-prod` — End-to-End-Probe des laufenden Stacks.

An einem Nachmittag fand er 5 reale, latente Bugs:

- `.editorconfig` fehlt im Build-Image → Build bricht mit `TreatWarningsAsErrors` ab
- Compose-Env-Casing `${EMBEDDINGS__APIKEY}` $\neq$ `Embeddings__ApiKey` → Embeddings still no-op
- Leerstring-Modellname → `AssertNotNullOrEmpty` -Crash beim Start
- Mistral lehnt das `dimensions` -Feld ab → 422
- Passbolt-Refs vom Makefile überschattet → KI-Call erreichte nie den Provider

*KI schrieb den Code. Eine **KI-gestützte Prüfung** fand, was die KI übersah.*

Der Mensch bleibt im Loop — als Reviewer.

Fazit

KI ist ein starker Accelerator für Infrastruktur-Code —

Boilerplate, Migrations, Tests entstehen in Minuten.

Sie braucht menschliche Führung bei Architektur & Domäne —

FK-Strategie, Performance, Scope, aktuelle Versionen.

Der Mensch bleibt Architekt und Reviewer.

Code & Doku: github.com/freaxnx01/FlowHub-CAS-AISE · Danke — Fragen?

Anhang · Abkürzungen

Abk.	Bedeutung	Abk.	Bedeutung
KI / AI	Künstliche Intelligenz	MEAI	Microsoft.Extensions.AI
SSR	Server-Side Rendering	LLM	Large Language Model
EF Core	Entity Framework Core (ORM)	pgvector	Postgres-Vektor-Erweiterung
HNSW	Approx.-Nearest-Neighbour-Index	ANN	Approximate Nearest Neighbour
DI	Dependency Injection	TDD	Test-Driven Development
RFC 9457	HTTP Problem Details	ADR	Architecture Decision Record
MVP	Minimum Viable Product	LTS	Long-Term Support
DMS	Dokumentenmanagement-System	OIDC	OpenID Connect (geplant)
FK	Foreign Key (Fremdschlüssel)	N+1	N+1-Query-Problem